迷你编码智能体：从任务到补丁

自行实现模型、工具与执行器的最小闭环；不使用现成 Agent 框架。

01 整体执行闭环

读取任务
保留题面与约束

→

模型决策
每轮一个 JSON 动作

→

执行工具
读写文件、运行测试

→

验证交付
真实日志与代码补丁

模型决定下一步做什么，程序负责参数检查、工具执行和状态更新。写入后重新验证；测试、诊断及计划全部完成，才允许正常结束。

02 四项基础能力与取舍

能力	实现方式	边界 / 取舍
用户交互	追加指令、查看计划、随时中断； 写入和命令默认确认，--auto 自动执行。	人工可干预，评测不等待输入。
上下文管理	固定保留题面和状态；基线对单条结果 保留头尾，超预算删除最早动作与结果。	4500 字符单条阈值； 15000 字符请求预算。
任务管理	子目标跟踪 todo / doing / done； 更新时固定标题和顺序。	执行先后仍依赖模型遵守， 最终由课程过程检查核验。
基础工具	完整读文件、写业务文件、执行命令； 另提供文件列表与计划更新。	只写允许文件，只运行完整 pytest 或题目诊断脚本。

03 轨迹与代码规模

run.log 实时记录完整请求、响应、文件内容和命令输出；上下文压缩不会删除原日志。patch.diff 根据初始快照生成，失败或中断也保存已有改动。

> **288 行核心代码 • 12 项机制测试通过**
>
> 核心包含 agent.py 的 SYSTEM、Agent 和全部 context_selection.py，按非空、非纯注释物理行计数；CLI、API 传输、日志和底层工具另计。环境：WSL 2 / Python 3.12.3 / pytest 8.4.2 / GNU patch 2.7.6。

特色功能：先选信息，再压缩

Diagnostic-Driven Context Selection and Compression / 确定性规则，无额外模型调用。

01 一次模型请求前，做四步处理

观察去重
更新命令结果；
写入使旧源码失效

→

诊断定位
识别当前阶段；
提取文件、测试、异常

→

相关性排序
验证、失败、路径
匹配与近期代码优先

→

预算内组装
摘录诊断行、去重；
超长内容再保留头尾

02 当前状态决定优先保留什么

状态	判断依据	优先信息
探索定位	尚未形成有效诊断	题面、文件列表、已读业务代码
修复失败	当前 pytest 或诊断返回失败	失败测试、异常位置、相关代码
修改后验证	已写入，尚未完成测试验证	最近改动、待验证项、历史失败参考
收尾	最近完整测试已通过	验证结果、计划完成状态

状态用于信息排序，不代替完成检查。收尾时仍须满足诊断通过、计划全部 done；写入前的失败不再作为当前诊断锚点。

03 预算和证据保持一致

固定保留	按相关性填入	始终保留在日志中
系统提示、完整题面、 追加要求、计划与验证状态	当前诊断摘录、相关源码、 最近修改和必要工具结果	完整原输出；候选分数、 选择原因与实际请求内容

长命令输出抽取失败、断言及汇总附近原文，合并重复行。识别失败时退回头尾裁剪；长代码暂未实现函数级切片。baseline 与 diagnostic 共享提示词、工具和字符预算，仅改变上下文构造方式。

> **同题、同模型、同预算，交替各跑 3 次**
>
> 主实验固定第五题，每轮使用新副本；再分别跑五题回归。deepseek-flash，温度 0，上下文 15000 字符，输出 4096 token，最多 60 步；智能体 600 秒、最终验证 120 秒。开关：--context-mode baseline|diagnostic。

实验结果：通过了，成本未下降

16 次真实运行全部保留；功能测试与过程检查分别报告，避免把代码正确等同于整题通过。

01 第五题对照：两组均为 3 / 3 通过

轮次	输入 token：基线 / 诊断	调用次数：基线 / 诊断	耗时秒：基线 / 诊断
1	60,648 / 79,956	15 / 28	36.8 / 89.8
2	83,301 / 73,170	19 / 29	42.4 / 84.2
3	71,984 / 69,161	15 / 20	30.7 / 62.8

平均输入量：基线 71,978 token，诊断 74,096 token（+2.9%）。诊断模式三轮调用均更多，本次未观察到稳定的成本收益。输入 token 包含重复上下文，并不等于实际费用。

02 五题回归：功能全过，过程仍有波动

公开题	基线：测试 / 过程	诊断：测试 / 过程
折扣修复	11/11 / 通过	11/11 / 通过
新增功能与回归	23/23 / 通过	23/23 / 通过
跨文件定位	14/14 / 通过	14/14 / 通过
净额修复 + CSV	10/10 / 未通过	10/10 / 通过
长日志定位	28/28 / 通过	28/28 / 通过

整题得分为基线 4/5、诊断 5/5。基线第四题合并写入两个子任务，跳过净额修复后的中间验证；最终 10 个功能测试虽通过，过程仍不合格。一次回归的差异不足以说明诊断模式更稳定。

03 如何理解这组结果

课程仓库小、修复点集中，基线可能已经保住足够线索，任务难度未必能体现智能选择的优势。但当前诊断模式确有重复调用，重组历史也可能削弱动作与结果的连贯性。这两种可能需要分开验证。

下一步可预先设计无关日志、中段关键错误和多文件依赖压力任务，再按相同预算比较；本次不追加有利样本。温度 0 也不保证完全复现，三轮仅支持小样本观察。

证据：experiments/feature/comparison01/ 下的 protocol.json、results.json 及各轮配置、完整日志、补丁。参考：课程 README、题目、日志规范和公开评测器。实现与报告由 AI 编程助手协助完成，数字来自真实 API 运行。